MON APPRENTISSAGE — SERIES PYTHON

Python

Du zéro à l'expert

Chapitre 8 — Les Listes

La structure de données la plus puissante de Python

Auteur :

KETOTSA AMÉVI CLAUDE

Date de publication :

6 mars 2026

Publié dans le cadre de mon parcours d'apprentissage Python

Table des matières

Chapitre 8 — Les Listes **2**
0.1 Créer une liste 2
0.2 Accéder aux éléments — Indexation et slicing 2
0.3 Ajouter des éléments 3
0.4 Supprimer des éléments 4
0.5 Rechercher dans une liste 5
0.6 Trier une liste 5
0.7 Copier une liste 6
0.8 Les listes imbriquées — Matrices 7
0.9 La list comprehension 7
0.10 Ce qui m'a le plus surpris dans ce chapitre 8
0.11 Résumé du Chapitre 8 9

Chapitre 8 — Les Listes

À retenir

La liste est la structure de données la plus utilisée en Python. Elle permet de **stocker plusieurs valeurs** dans une seule variable, dans un ordre précis, et de les modifier à volonté. Contrairement aux chaînes, les listes sont **mutables** — on peut les modifier après leur création. C'est ici que Python commence à révéler toute sa puissance.

Créer une liste

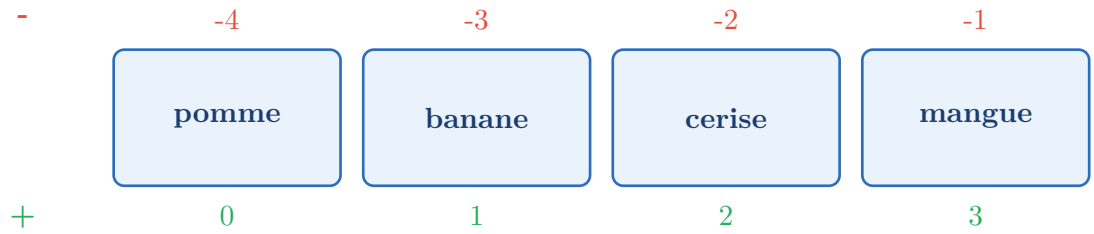
Une liste est délimitée par des **crochets** `[]` et ses éléments sont séparés par des virgules.

Créer des listes

```python
# Liste de chaines
fruits = ["pomme", "banane", "cerise", "mangue"]

# Liste de nombres
notes = [15, 18, 12, 9, 17]

# Liste mixte     Python accepte des types differents
mixte = [42, "Python", 3.14, True, None]

# Liste vide
panier = []

# Liste avec list()
lettres = list("Python")
print(lettres)   # ['P', 'y', 't', 'h', 'o', 'n']

# Verifier la taille
print(len(fruits))   # 4
print(len(notes))    # 5
```

Accéder aux éléments — Indexation et slicing

Comme les chaînes, les listes utilisent des **index** qui commencent à 0.



Accéder et découper une liste

```python
fruits = ["pomme", "banane", "cerise", "mangue"]

# Acces par index
print(fruits[0])     # pomme
print(fruits[-1])    # mangue
print(fruits[2])     # cerise

# Slicing
print(fruits[1:3])   # ['banane', 'cerise']
print(fruits[:2])    # ['pomme', 'banane']
print(fruits[2:])    # ['cerise', 'mangue']
print(fruits[::-1]) # ['mangue', 'cerise', 'banane', 'pomme']

# Modifier un element      les listes sont MUTABLES
fruits[1] = "fraise"
print(fruits)    # ['pomme', 'fraise', 'cerise', 'mangue']
```

Erreur fréquente

Contrairement aux chaînes, les listes sont **mutables** — on peut modifier un élément directement avec `liste[index] = nouvelle_valeur`. C'est la différence fondamentale entre `str` et `list` en Python.

Ajouter des éléments

append(), insert(), extend()

```python
fruits = ["pomme", "banane"]

# append()      ajouter a la FIN
fruits.append("cerise")
print(fruits)    # ['pomme', 'banane', 'cerise']

# insert()      ajouter a une POSITION precise
fruits.insert(1, "fraise")
print(fruits)    # ['pomme', 'fraise', 'banane', 'cerise']

# extend()      ajouter PLUSIEURS elements
nouveaux = ["mangue", "kiwi"]
fruits.extend(nouveaux)
print(fruits)
# ['pomme', 'fraise', 'banane', 'cerise', 'mangue', 'kiwi']

# + pour concatener deux listes (cree une NOUVELLE liste)
liste1 = [1, 2, 3]
liste2 = [4, 5, 6]
```

```python
liste3 = liste1 + liste2
print(liste3)    # [1, 2, 3, 4, 5, 6]

# * pour repeter
zeros = [0] * 5
print(zeros)     # [0, 0, 0, 0, 0]
```

Ce qui m'a résisté

La différence entre `append()` et `extend()` m'a piégé plusieurs fois. `append([4, 5])` ajoute la liste entière comme **un seul élément** — on obtient une liste dans une liste. `extend([4, 5])` ajoute chaque élément **individuellement**. Résultat très différent, erreur très subtile à déboguer.

Supprimer des éléments

remove(), pop(), del, clear()

```python
fruits = ["pomme", "banane", "cerise", "banane", "mangue"]

# remove()      supprimer la PREMIERE occurrence d'une valeur
fruits.remove("banane")
print(fruits)    # ['pomme', 'cerise', 'banane', 'mangue']

# pop()      supprimer et RETOURNER l'element a un index
# (dernier element par defaut)
dernier = fruits.pop()
print(dernier)    # mangue
print(fruits)     # ['pomme', 'cerise', 'banane']

element = fruits.pop(1)
print(element)    # cerise
print(fruits)     # ['pomme', 'banane']

# del      supprimer par index ou slice
nombres = [10, 20, 30, 40, 50]
del nombres[2]
print(nombres)    # [10, 20, 40, 50]

del nombres[1:3]
print(nombres)    # [10, 50]

# clear()      vider completement la liste
fruits.clear()
print(fruits)     # []
```

Rechercher dans une liste

in, index(), count()

```python
notes = [15, 18, 12, 18, 9, 17, 12]

# in      verifier la presence
print(18 in notes)     # True
print(20 in notes)     # False

# index()      trouver la position de la PREMIERE occurrence
print(notes.index(18))    # 1
print(notes.index(12))    # 2

# count()      compter les occurrences
print(notes.count(18))    # 2
print(notes.count(12))    # 2
print(notes.count(5))     # 0

# min(), max(), sum()      statistiques rapides
print(min(notes))     # 9
print(max(notes))     # 18
print(sum(notes))     # 101
moyenne = sum(notes) / len(notes)
print(f"Moyenne : {moyenne:.2f}")    # 14.43
```

Trier une liste

sort() et sorted()

```python
nombres = [3, 1, 4, 1, 5, 9, 2, 6]

# sort()      trie la liste EN PLACE (modifie l'original)
nombres.sort()
print(nombres)    # [1, 1, 2, 3, 4, 5, 6, 9]

nombres.sort(reverse=True)
print(nombres)    # [9, 6, 5, 4, 3, 2, 1, 1]

# sorted()      retourne une NOUVELLE liste triee
originale = [3, 1, 4, 1, 5, 9]
nouvelle = sorted(originale)
print(originale)    # [3, 1, 4, 1, 5, 9]  (inchangee)
print(nouvelle)     # [1, 1, 3, 4, 5, 9]

# Trier des chaines
prenoms = ["Charlie", "Alice", "Bob", "David"]
prenoms.sort()
print(prenoms)    # ['Alice', 'Bob', 'Charlie', 'David']
```

```python

# Trier par longueur avec key=
mots = ["python", "est", "un", "langage", "puissant"]
mots.sort(key=len)
print(mots)    # ['un', 'est', 'python', 'langage', 'puissant']

# reverse()      inverser l'ordre
nombres = [1, 2, 3, 4, 5]
nombres.reverse()
print(nombres)    # [5, 4, 3, 2, 1]
```

Ce qui m'a résisté

La distinction entre `sort()` et `sorted()` est cruciale. `sort()` modifie la liste **en place** et ne retourne **rien** (`None`). J'ai fait l'erreur d'écrire `liste = liste.sort()` — résultat : ma variable valait `None`. `sorted()` est plus sûr quand on veut conserver l'original. Cette nuance entre méthodes *in place* et fonctions qui retournent une valeur est un pattern récurrent en Python.

Copier une liste

Copie superficielle vs référence

```python
#      PIEGE      assignation cree une REFERENCE, pas une copie
liste_a = [1, 2, 3]
liste_b = liste_a       # liste_b pointe vers le MEME objet

liste_b.append(4)
print(liste_a)    # [1, 2, 3, 4]       liste_a est modifiee !
print(liste_b)    # [1, 2, 3, 4]

#      Copie avec copy()
liste_a = [1, 2, 3]
liste_b = liste_a.copy()

liste_b.append(4)
print(liste_a)    # [1, 2, 3]       inchangee
print(liste_b)    # [1, 2, 3, 4]

#      Autres facons de copier
liste_c = liste_a[:]          # slicing complet
liste_d = list(liste_a)       # constructeur list()

print(liste_c)    # [1, 2, 3]
print(liste_d)    # [1, 2, 3]
```

Ce qui m'a résisté

Le comportement de `liste_b = liste_a` m'a complètement surpris — et piégé. En Python, on n'assigne pas une valeur, on assigne une **référence**. Les deux variables pointent vers le **même objet en mémoire**. Modifier l'une modifie l'autre. Ce concept de référence vs copie est l'une des subtilités les plus importantes de Python, et l'une des plus sournoises à déboguer quand on ne la connaît pas.

Les listes imbriquées — Matrices

Une liste peut contenir d'autres listes, ce qui permet de créer des **structures à deux dimensions**.

Listes imbriquées et matrices

```python
# Matrice 3x3
matrice = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

# Acceder a un element : matrice[ligne][colonne]
print(matrice[0][0])   # 1  (ligne 0, colonne 0)
print(matrice[1][2])   # 6  (ligne 1, colonne 2)
print(matrice[2][1])   # 8  (ligne 2, colonne 1)

# Parcourir une matrice
for ligne in matrice:
    for valeur in ligne:
        print(f"{valeur:3}", end="")
    print()
#      1  2  3
#      4  5  6
#      7  8  9

# Modifier un element
matrice[1][1] = 99
print(matrice[1])   # [4, 99, 6]

# Creer une grille 4x4 remplie de zeros
grille = [[0] * 4 for _ in range(4)]
print(grille)
```

La list comprehension

La **list comprehension** est une syntaxe élégante et pythonique pour créer des listes en une seule ligne.

List comprehension — syntaxe de base

```python
# Syntaxe : [expression for element in iterable if condition]

# Sans list comprehension
carres = []
for i in range(1, 6):
    carres.append(i ** 2)
print(carres)   # [1, 4, 9, 16, 25]

# Avec list comprehension     meme resultat, une ligne
carres = [i ** 2 for i in range(1, 6)]
print(carres)   # [1, 4, 9, 16, 25]

# Avec condition     filtrer les pairs
pairs = [i for i in range(1, 21) if i % 2 == 0]
print(pairs)   # [2, 4, 6, 8, 10, 12, 14, 16, 18, 20]

# Transformer des chaines
fruits = ["pomme", "banane", "cerise"]
majuscules = [f.upper() for f in fruits]
print(majuscules)   # ['POMME', 'BANANE', 'CERISE']

# Longueurs des mots
longueurs = [len(f) for f in fruits]
print(longueurs)   # [5, 6, 6]

# Filtrer et transformer
notes = [15, 8, 18, 5, 12, 3, 17]
admis = [n for n in notes if n >= 10]
print(admis)   # [15, 18, 12, 17]
```

À retenir

La list comprehension est considérée comme **pythonique** — c'est la façon idiomatique de créer des listes transformées ou filtrées. Elle est plus lisible et souvent plus rapide qu'une boucle `for` équivalente. C'est l'une des syntaxes qui distingue vraiment Python des autres langages.

Ce qui m'a le plus surpris dans ce chapitre

Ce qui m'a résisté

Trois révélations de ce chapitre :

1. `liste_b = liste_a` ne copie rien. C'est probablement la subtilité la plus dangereuse que j'ai rencontrée jusqu'ici. Deux variables qui pointent vers le même objet en mémoire — modifier l'une modifie l'autre silencieusement. Sans connaître ce comportement, on peut passer des heures à chercher un bug qui n'a aucun sens

apparent. La solution `.copy()` ou `[::]` est simple, mais encore faut-il savoir qu'on en a besoin.

2. `sort()` retourne `None`. J'ai écrit `ma_liste = ma_liste.sort()` et perdu ma liste entière. `sort()` modifie la liste *en place* et retourne `None`. Cette distinction entre les méthodes qui modifient l'objet sans rien retourner et les fonctions qui retournent une nouvelle valeur est un concept fondamental que Python applique de façon cohérente — mais qui surprend au début.

3. La list comprehension — une ligne pour tout faire. Voir qu'on peut filtrer, transformer et créer une liste entière en une seule expression lisible m'a révélé un nouveau niveau d'expressivité. Ce n'est pas juste du sucre syntaxique — c'est une façon de *penser* différemment la transformation des données. Python encourage à décrire **ce qu'on veut** plutôt que **comment le faire**.

Résumé du Chapitre 8

Ce que j'ai appris dans ce chapitre :

- Créer des listes avec `[]` — mutables, ordonnées, hétérogènes.
- Accéder avec l'**indexation** et découper avec le **slicing**.
- Ajouter : `append()`, `insert()`, `extend()`.
- Supprimer : `remove()`, `pop()`, `del`, `clear()`.
- Trier : `sort()` modifie en place, `sorted()` retourne une nouvelle liste.
- Copier : toujours utiliser `.copy()` — jamais `b = a`.
- Listes imbriquées : accès avec `matrice[ligne][colonne]`.
- **List comprehension** : `[expr for x in liste if condition]`.

"La liste est le couteau suisse de Python. Quand on la maîtrise, on peut tout construire."